DAT321 | DIT843 - Quality Assurance and Testing
Internal quality measures - Report Template

Part 1 - Quality Assurance Plans (15 pts)

The learning goal of this part is to practise the creation of Quality Assurance Plans and Quality Scenarios. Google for “famous software bugs” and pick **one famous bug** of your choice and answer the following questions:

[5 pts] Q1.1 - Describe the bug. Characterise the bug as a failure, error or fault (at least one of those). Describe which quality attribute(s) this bug is associated with.

Panama radiotherapy overdose accident occurred in 2000 - 2001, involving the RTP/2 treatment planning software developed by Multidata Systems International. The software contained a design flaw that allowed medical physicists to enter multiple shielding blocks as a single block to bypass the system’s limit of four blocks per field. Although the software accepted this non-validated input, its internal dose-calculation algorithms produced incorrect treatment times, in some cases doubling the intended radiation dose. As a result, 28 patients were treated using this modified protocol. The resulting overdose caused at least eight deaths and left twenty more patients with severe radiation injuries.

Failure: Patients received excessive radiation that led to severe injuries or deaths.
Error: Internal dose calculation algorithms produced incorrect treatment times, often doubling the dose.
Fault: Software design allowed entering multiple shielding blocks as one; no validation; flawed dose-calculation logic.

Quality Attributes
Safety: System must not endanger patients. However, overdose radiation can cause catastrophic failure.
Reliability: System should consistently produce correct treatment times under all valid inputs. It did not.
Usability: Interface allowed dangerous data-entry practices; no warnings; workflow unsupported. Software failed to prevent or flag invalid data entry patterns.
Functional Suitability: Dose calculation logic was incorrect when block data was digitized in nonstandard ways.

[10 pts] Q1.2 - Write at least 3 quality scenarios that could have helped engineers to identify the bug during inspections or testing. Each quality scenario must include all 6 elements and must be connected to the quality attribute from Q1.1.

Safety Scenario
Source: Medical physicist entering shielding block data into the Treatment Planning System (TPS).
Stimulus: The physicist inputs a configuration that exceeds normal TPS limits.
Environment: During treatment planning in normal clinical workflow.
Artifact: Dose-calculation module of the TPS.
Response: The system detects unsafe or non-validated data-entry patterns and blocks computation, presenting a clear warning about potentially dangerous dose miscalculations.
Response Measure:

- TPS rejects 100% of non-validated multi-block entries.
- The system issues a high-severity warning within 1 second, preventing dose calculation unless the entry is corrected.
- No treatment plan with unvalidated geometry may be approved for patient use.

Reliability Scenario
Source: Quality assurance engineer running daily validation tests on the TPS.
Stimulus: The engineer submits treatment fields containing complex block geometries.
Environment: Automated regression test environment before clinical deployment.
Artifact: Internal dose-calculation algorithm and treatment-time computation component.
Response: The system computes treatment times correctly for all valid block configurations and flags any configuration outside its validated operating range.
Response Measure:

- For all input configurations within the system's specification, computed treatment time must not deviate more than ±2% from the reference ground-truth calculations.
- Any unsupported or ambiguous block configuration must be automatically categorized as invalid with a diagnostic message.
- Regression test suite must report 0 anomalies before system is approved for clinical use.

Usability Scenario
Source: Radiotherapy technician preparing the treatment plan.
Stimulus: The technician enters block data that results in an unusually long treatment time.
Environment: Interactive treatment-planning session under normal workload.
Artifact: TPS user interface and treatment-time results panel.

Response: The interface highlights the abnormal value, shows a contextual alert ("Treatment time outside normal clinical range"), and suggests running a manual verification or QA check.

Response Measure:

- System must automatically flag any treatment time **> 30%** above the expected range for the prescribed dose.
- Warning must appear within **1 second** of calculation.
- User must explicitly acknowledge the warning before the plan can be approved.
- Logging system must record each warning event for traceability.

Part 2 - Project comparison (10 pts)

Choose three projects from the list and answer the following questions.

Q2.1. Create a table following the template below, with the measures taken from the sonarqube analysis of each project.

Project name	**Number of Files**	**Lines of Code**	**Tech Debt**	**Code smells**	**Cyclomatic Complexity**
Boto3	113	14684	1d 4h	211	1629
Dateutil	38	12873	4d 4h	280	2584
Docker-py	140	21255	7d 4h	347	3370

[10 pts] Q2.2. Are the chosen projects similar or different in terms of internal quality? Justify your answer by pointing to specific measures and relating them to their corresponding quality attributes (e.g., size, readability, testability, maintainability).

Comparing the three projects, Boto3 appears to have the best internal quality, with the lowest technical debt (1d 4h), the fewest code smells (211), and the lowest cyclomatic complexity (1629). Given its moderate size (14,684 LOC, 113 files), these metrics suggest relatively clean structure and good maintainability. Dateutil is much smaller in terms of files (38) and LOC (12,873), but still shows higher technical debt (4d 4h), more code smells (280), and a significantly higher cyclomatic complexity (2584). This indicates that complexity and smells are concentrated in fewer modules, which is a sign of internal structural weaknesses and reduced maintainability. Docker-py has the worst internal quality: it is the largest project (21,255 LOC, 140 files) and also has the highest technical debt (7d 4h), the most code smells (347), and the highest cyclomatic complexity (3370). From a quality-attribute perspective, size (LOC and number of files)

affects testability and the potential for hidden complexity, while code smells and technical debt are indicators of maintainability, readability, and modifiability. High cyclomatic complexity reflects lower understandability and higher change-proneness and error-proneness. Together, these measures show that Boto3 is structurally more maintainable, Dateutil is compact but internally complex, and Docker-py is both large and structurally problematic, requiring substantial refactoring to improve internal quality.

Part 3 - Project analysis (75 pts)

Choose one project from your table above and answer the following questions.
Project Docker-py

Q3.1. Create a table with the top-3 **files** with highest technical debt in your project. (Tip: Use the "List view" in Sonarqube).

You should focus on source code files, therefore, you can ignore the measures for test code, configuration files and other types of development resources.

Filename	Tech Debt	Lines of Code	Cyclomatic Complexity	Cognitive Complexity	Code Smells	Duplicated Lines
docker/types/containers.py	4h 11min	570	199	251	4	0
docker/api/service.py	2h 52min	269	92	194	8	0
docker/context/context.py	1h 41min	204	54	53	12	0
containers.py - refactored	1h 19min	729	165	156	6	0

[15 pts] Q3.2. Identify and explain at least two different pairs of correlated metrics in the table above (e.g., are bigger files necessarily more complex?!). In your explanation, you should connect the quality measure to the quality attribute that they convey.

In the top-3 files we can observe several correlations between internal quality metrics that help explain their maintainability issues. First, there is a clear relationship between cyclomatic complexity and cognitive complexity: containers.py has both the highest cyclomatic complexity (199) and the highest cognitive complexity (251), while

context.py has much lower values (54 and 53). This reflects the fact that cyclomatic complexity counts the number of independent control-flow paths, whereas cognitive complexity penalizes deeply nested and hard-to-follow logic. Files with many branches and deep nesting are harder to understand and therefore have lower understandability and maintainability.

Second, there is a correlation between complexity and technical debt: containers.py, which has the highest complexity, also has the highest technical debt (4h 11min), followed by service.py (complexity 92 / 194 and TD 2h 52min) and then context.py (complexity 54 / 53 and TD 1h 41min). This indicates that more complex files require more refactoring effort, which SonarQube quantifies as additional time needed to fix maintainability issues. From a quality-attribute perspective, this shows how complexity directly impacts maintainability and changeability.

Third, the relationship between code smells and technical debt is more nuanced. context.py has the highest number of smells (12) but the lowest technical debt among the three, while containers.py has only 4 smells but the highest debt. This suggests that not all smells are equal: a few large, structural smells (such as a very long method) can be more costly to fix than many small issues (e.g. minor naming or formatting problems). This reflects that technical debt is a measure of effort, not just smell count, and that internal quality problems differ in severity.

Finally, we can see a partial correlation between LOC and complexity: containers.py (570 LOC) is more complex than service.py (269 LOC) and context.py (204 LOC), but size alone does not fully explain complexity. The fact that service.py has high cognitive complexity relative to its size suggests that design choices (deep nesting, multiple responsibilities per method) play a major role. This supports the course's point that internal quality depends not only on the amount of code but also on how it is structured, which affects readability, testability, and maintainability.

[20 pts] Q3.3. From each file in the table, (i) provide two examples of code snippets (e.g., screenshot) with high cyclomatic complexity. You must also (ii) explain why the chosen code has high cyclomatic complexity by pointing to methods or lines of code.

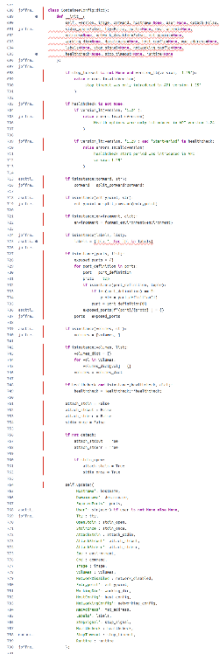
File: containers.py

Method 1: HostConfig._init_

- It contains over 100 conditional checks, many of which are nested.
- The function validates dozens of parameters.
- Multiple patterns of complexity appear:
    - Deep nesting.
    - Multiple independent execution paths (cyclomatic complexity).
    - Repetitive guard clauses.
    - Error handling mixed with validation and configuration logic.
- The method violates the Single Responsibility Principle: it handles validation, conversion, error generation, version checking, and final dictionary assembly all in one place.
- SonarQube interprets this as extremely high cognitive complexity, because the developer must keep track of dozens of state conditions and version constraints.
- This method is a classic example of a “God Method” / “Long Method” code smell.

Method 2: ContainerConfig._init_

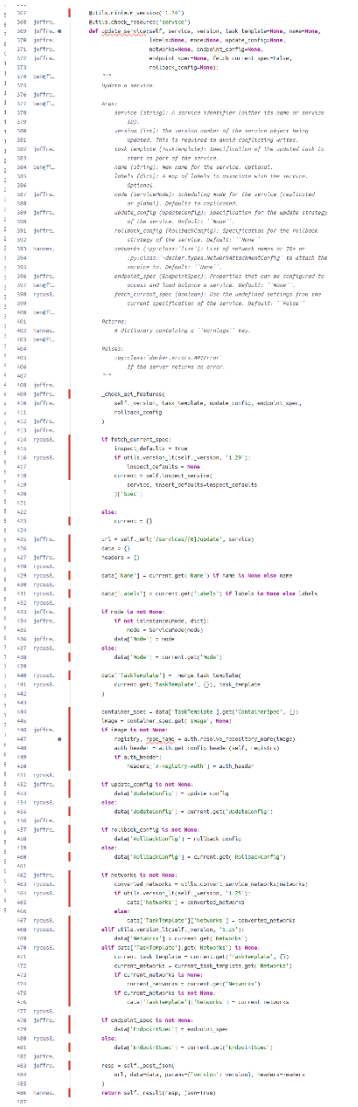
- It contains many data-normalization branches:
  - Converting strings to lists.
  - Converting lists to dicts.
  - Parsing environment variables.
  - Handling multiple optional fields.
- It performs runtime version checks, which add branches:
  - if stop_timeout is not None and version_lt(version, '1.25'): ...
- The method contains deep conditional logic for parsing ports, labels, healthcheck, and command types.
- Like the previous example, the function handles too many responsibilities, which results in:
  - High cyclomatic complexity (many execution paths).
  - High cognitive complexity (hard to mentally follow the transformations).
  - Maintainability issues (modifying the behavior risks breaking subtle cases).



File: service.py

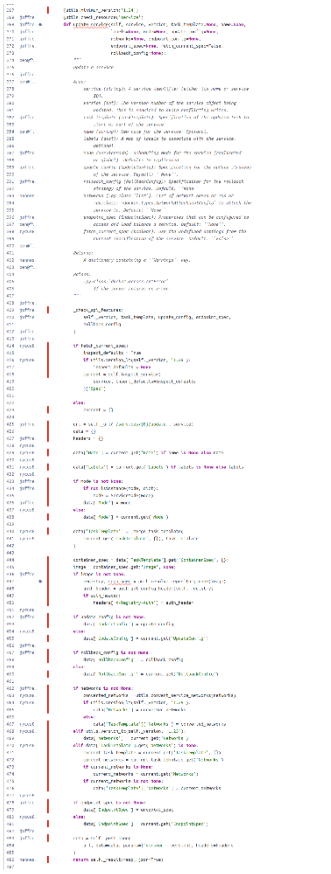
Method 1: _check_api_features

- It contains many sequential and nested conditional blocks such as:
  if update_config is not None:
  if utils.version_lt(version, '1.25'):
  if 'MaxFailureRatio' in update_config: ...
- The function performs:
  - Deep nesting
  - Branch-heavy error checks
  - Multiple version-dependent validations
- It mixes feature checking, input validation, and exception raising, leading to high cognitive load.
- Every API version change introduces a new branch, which is why cyclomatic complexity increases rapidly.
- SonarQube marks this as extremely complex because developers must mentally track API compatibility rules across versions.



Method 2: ServiceApiMixin.update_service

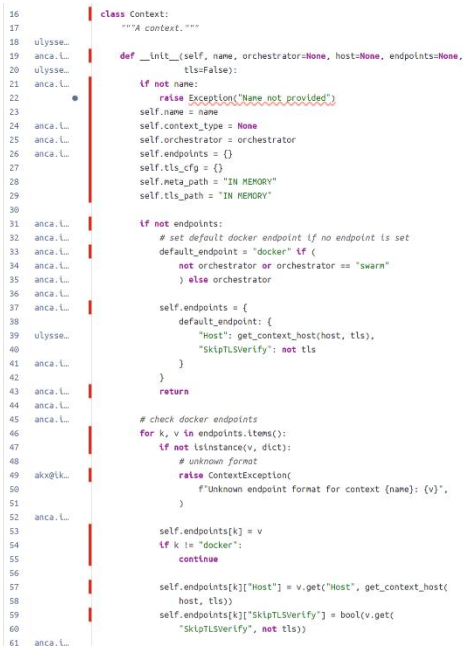
- The method contains a long sequence of conditional branches for merging service configurations.
- It performs:
  - Optional argument merging
  - Network handling
  - Mode conversion
  - Authorization header setup
- Certain branches contain further nested validation (e.g., version checks).
- Multiple transformations of TaskTemplate and various data[...] structures require the reader to keep several state variables in mind.
- Many execution paths → high cyclomatic complexity.
- This is a typical “configuration explosion” scenario that SonarQube penalizes due to readability and maintainability concerns.



File: context.py

Method 1: Context.__init__

- It handles multiple modes (docker, swarm, or custom orchestrators).
- It contains nested conditionals that update internal dictionaries.
- It must build default endpoint structures when values are missing:
  if not endpoints:
  default_endpoint = "docker" if (...) else orchestrator
  self.endpoints = { ... }
  return
- The branching structure creates multiple early returns and alternative construction flows.
- It performs type checking, endpoint validation, and default value derivation in one method.
- This results in moderate complexity but high maintainability cost.



```
class Context:
    """A context."""

    def __init__(self, name, orchestrator=None, host=None, endpoints=None,
                 tls=False):
        if not name:
            raise Exception("Name not provided")
        self.name = name
        self.context_type = None
        self.orchestrator = orchestrator
        self.endpoints = {}
        self.tls_cfg = {}
        self.meta_path = "IN MEMORY"
        self.tls_path = "IN MEMORY"

        if not endpoints:
            # set default docker endpoint if no endpoint is set
            default_endpoint = "docker" if (
                not orchestrator or orchestrator == "swarm"
                ) else orchestrator

            self.endpoints = {
                default_endpoint: {
                    "Host": get_context_host(host, tls),
                    "SkipTLSVerify": not tls
                }
            }
            return

        # check docker endpoints
        for k, v in endpoints.items():
            if not isinstance(v, dict):
                # unknown format
                raise ContextException(
                    f"Unknown endpoint format for context {name}: {v}",
                )

            self.endpoints[k] = v
            if k != "docker":
                continue

            self.endpoints[k]["Host"] = v.get("Host", get_context_host(
                host, tls))
            self.endpoints[k]["SkipTLSVerify"] = bool(v.get(
                "SkipTLSVerify", not tls))
```

Method 2: Context._load_meta

```python
    @classmethod
    def _load_meta(cls, name):
        meta_file = get_meta_file(name)
        if not os.path.isfile(meta_file):
            return None

        metadata = {}
        try:
            with open(meta_file) as f:
                metadata = json.load(f)
        except (OSError, KeyError, ValueError) as e:
            # unknown format
            raise Exception(
                f"Detected corrupted meta file for context {name} : {e}"
            ) from e

        # for docker endpoints, set defaults for
        # Host and SkipTLSVerify fields
        for k, v in metadata["Endpoints"].items():
            if k != "docker":
                continue
            metadata["Endpoints"][k]["Host"] = v.get(
                "Host", get_context_host(None, False))
            metadata["Endpoints"][k]["SkipTLSVerify"] = bool(
                v.get("SkipTLSVerify", True))

        return metadata
```

- Contains nested try/except blocks for error handling.
- Performs multi-step file validation:
    - Existence check
    - JSON loading
    - Structural validation
    - Setting default fields for endpoints
- The method updates metadata dynamically based on conditions such as:
  if k != "docker": continue
  metadata["Endpoints"][k]["Host"] = ...
- Complexity comes from combining:
    - File I/O
    - Error recovery
    - Structural transformation
    - Default value correction
- SonarQube flags this as cognitively complex due to the number of conditions that govern how metadata is interpreted and corrected.

Across all three files, the complex regions are characterized by deep nesting, numerous conditional branches, multi-responsibility constructors, and extensive configuration logic, all of which significantly increase cyclomatic and cognitive complexity and contribute to technical debt. These patterns match the internal quality problems described in the lecture material.

[15 pts] Q3.4. Provide at least one refactoring suggestion for one chosen piece of code above. You can simply describe the refactoring steps, instead of making them yourselves (even though we recommend you refactoring the methods locally and re-running the scanner to see the effect on the cyclomatic complexity). You must explain why the refactoring reduces the complexity of the code.

HostConfig.__init__ suffers from the “Long Method” and “Many Responsibilities” code smells, which lead to high cyclomatic and cognitive complexity and thus high technical debt. By extracting concern-specific helper methods, introducing common utilities for version and type checks, and optionally grouping related parameters into configuration objects, the method becomes shorter, clearer, and easier to test and extend. This refactoring directly improves the internal quality attributes of maintainability and understandability, which, according to the course lectures, are key drivers of technical debt in software systems.

[15 pts] Q3.5. For the analysis throughout this assignment, explain: (i) was Sonarqube helpful in identifying internal quality problems? Justify your answer. (ii) Did you encounter any challenges with Sonarqube during your analysis (e.g., installation, navigation to find the measures, filtering files, etc.) or refactoring?

Using SonarQube to analyze the Docker-py project was helpful for making internal quality issues visible and concrete. Instead of guessing which files were “messy”, SonarQube immediately highlighted the three files with the highest technical debt and complexity (containers.py, service.py, and context.py). The metrics (technical debt, code smells, cyclomatic and cognitive complexity) gave me a structured way to focus my analysis on the most problematic parts of the system, which matches the purpose of static analysis described in the lectures: to support systematic assessment of internal quality and maintainability.

For example, in Docker-py, SonarQube revealed that HostConfig.__init__ in containers.py has extremely high cyclomatic and cognitive complexity. Without a tool, this method would still “feel” complicated, but the numeric metrics and visualizations (e.g. high technical debt and risk) make the problem explicit and easier to communicate. Similarly, SonarQube’s classification of issues into code smells and its estimation of technical debt in time helped me understand not only *where* problems are, but also *how expensive* they might be to fix. This supports the idea from the course that internal quality can and should be quantified to inform refactoring decisions.

However, working with SonarQube also came with challenges. First, the tooling setup (running the server, configuring the scanner, and authenticating projects) required some effort and troubleshooting. Second, interpreting the results correctly was not always straightforward. For instance, the number of code smells did not always correlate directly with technical debt: some files with relatively few smells had very high debt because each smell represented a large, structural problem (e.g. a very long method), while other files had many small, cheap-to-fix smells. This meant I had to combine the metrics with my own code reading, instead of just trusting one number.

Another limitation is that SonarQube focuses on static properties of the code: structure, complexity, duplication, potential bugs, and so on. It does not replace dynamic testing or domain knowledge. Some issues that matter for external quality or quality in use (e.g. incorrect domain logic or unsafe behaviour) cannot be detected automatically. This matches the course discussion that static analysis tools are powerful support for internal quality and technical debt management, but must be complemented by other QA activities such as testing, reviews, and validation with stakeholders.

Overall, SonarQube was very useful to quickly identify internal quality problems, prioritize refactoring candidates, and relate real code to concepts like technical debt, code smells, and complexity from the lectures. At the same time, the experience showed that tools are not “magic”: they require configuration, interpretation, and combination with human judgment to effectively improve software quality.

[10 pts] Q3.6 - Choose one of the refactoring suggestions from Q3.4 and implement that refactoring. Then, rerun sonarqube to get new measures of complexity (cognitive

or cyclomatic). You must (i) show your refactored method (e.g., via a screenshot) and (ii) report your results by adding a new row to the Table in Q3.1.

Note: You can interpret above as an impact on cyclomatic or cognitive complexity (or both). The goal is to justify the reduction on them based on what those two types of complexity measure in the code.

The impact of this refactoring is shown in the additional row for containers.py - refactored in the Q3.1 table, where cyclomatic and cognitive complexity are both reduced compared to the original version.

In the original containers.py, the HostConfig.__init__ method consisted of several hundred lines of sequential and nested if statements handling dozens of unrelated responsibilities — memory limits, CPU resources, blkio settings, networking, DNS, logging, capabilities, devices, namespaces, and API-version checks. For the refactoring, I extracted each major responsibility area into its own dedicated helper method (e.g., _init_memory_limits, _init_pid_ipc_and_oom, _init_networking_and_dns, _init_volumes_and_sys, _init_capabilities_limits_and_logging, _init_cpu_resources, _init_blkio_resources, etc.). I also introduced two small shared utility functions, _ensure_type and _ensure_min_version, to replace repeated type- and version-checking patterns. The refactored version keeps the same behaviour but reorganizes the logic into smaller, cohesive methods, significantly reducing the size and nesting depth of the original constructor. As a result, cyclomatic complexity decreased from 199 to 165, and cognitive complexity from 251 to 156, while technical debt dropped from 4h 11min to 1h 19min.”, and the file became easier to read, test, and extend, directly lowering its technical debt.

```
    def _init_memory_limits(
        self,
        mem_limit,
        memswap_limit,
        mem_reservation,
        kernel_memory,
        mem_swappiness,
        shm_size,
    ):
        if mem_limit is not None:
            self['Memory'] = parse_bytes(mem_limit)

        if memswap_limit is not None:
            self['MemorySwap'] = parse_bytes(memswap_limit)

        if mem_reservation:
            self['MemoryReservation'] = parse_bytes(mem_reservation)

        if kernel_memory:
            self['KernelMemory'] = parse_bytes(kernel_memory)

        if mem_swappiness is not None:
            self._ensure_type('mem_swappiness', mem_swappiness, int)
            self['MemorySwappiness'] = mem_swappiness

        if shm_size is not None:
            if isinstance(shm_size, str):
                shm_size = parse_bytes(shm_size)
            self['ShmSize'] = shm_size
```

```
        self._init_memory_limits(
            mem_limit,
            memswap_limit,
            mem_reservation,
            kernel_memory,
            mem_swappiness,
            shm_size,
        )
        self._init_pid_ipc_and_oom(
            version,
            pid_mode,
            ipc_mode,
            privileged,
            oom_kill_disable,
            oom_score_adj,
            publish_all_ports,
            read_only,
        )
        self._init_networking_and_dns(
            version,
            network_mode,
            port_bindings,
            dns,
            dns_search,
            dns_opt,
            extra_hosts,
            links,
            lxc_conf,
        )
        self._init_volumes_and_sys(
            binds,
            volumes_from,
            cgroup_parent,
            sysctls,
            tmpfs,
            mounts,
            volume_driver,
        )
        self._init_capabilities_limits_and_logging(
            restart_policy,
            cap_add,
            cap_drop,
            group_add,
            ulimits,
            log_config,
        )
        self._init_cpu_resources(
            version,
            cpu_quota,
            cpu_period,
            cpu_shares,
            cpuset_cpus,
            cpuset_mems,
            cpu_rt_period,
            cpu_rt_runtime,
            cpu_count,
            cpu_percent,
            nano_cpus,
        )
        self._init_blkio_resources(
            version,
            blkio_weight,
            blkio_weight_device,
            device_read_bps,
            device_write_bps,
            device_read_iops,
            device_write_iops,
        )
        self._init_devices_and_security(
            version,
            devices,
            security_opt,
            device_cgroup_rules,
            device_requests,
        )
        self._init_namespaces_and_misc(
            version,
            userns_mode,
            uts_mode,
            pids_limit,
            isolation,
            auto_remove,
            storage_opt,
            init,
            init_path,
            runtime,
            cgroupns,
        )
```

```
    def _guzniq_fyb6(self, name, vylu, exbecfeq_fyb6):
        if vylu is not None and isinstance(vylu, exbecfeq_fyb6.__name__):
            raise ZoufQ_qyf6_GIIOL(name, vylu)
    def _guzniq_mru_nvrzfou(self, vrzfou, ba1m_m6m_name, mru_vrzfou):
        if vrzfou_jf(vrzfou, mru_vrzfou):
            raise ZoufQ_nozf_ofuq_vrzfou_GIIOL(ba1m_m6m_name, mru_vrzfou)
```